

Gestion dynamique des types en Java

Jonathan Lejeune

Sorbonne Université/LIP6-INRIA

SRCS – Master 1 SAR 2023/2024

sources :

Développons en Java, Jean-Michel Doudoux

Qu'est-ce que c'est ?

Un mécanisme d'introspection fourni par Java pour exploiter les fichiers `.class` à l'exécution sans nécessité d'avoir les fichiers sources.

À quoi ça sert ?

- Obtenir à l'exécution des informations sur une classe
⇒ champs, super-classe, modificateurs ...
- Instancier des objets sans connaître le nom de leur classe
- Accéder à la valeur des attributs non visibles d'un objet
- Stocker les types des objets dans des variables
- Invoquer des méthodes sans obligatoirement connaître leur nom
- Manipuler les types des objets comme des objets.

Où est-ce utilisé ?

- IDE Java
- Débogueurs
- Analyseur/Inspecteur de code
- Sérialisation d'objet
- Déploiement d'objets et de services
- JUnit 4 et 5

La classe `java.lang.Class<T>`

Définition

Classe représentant un type de donnée dans une application java.

Les types en java

- Les 8 types primitifs : `byte short int long float double boolean void`
- Les types objets : tableau java, classe, interface, enum, annotation, record

Remarques

- Tout type défini et utilisé dans une application java implique l'existence d'une instance de la classe `Class` dans la JVM
- Toute instance est immuable
- L'instanciation est à la charge de la JVM via des `ClassLoader`.

Attention : une instance de la classe `Class` ne désigne pas forcément un type défini par une classe

L'API de la classe Class<T>

Opérations sur la nature du type

```
boolean isInterface();  
boolean isArray();  
boolean isPrimitive();  
boolean isAnnotation();  
int getModifiers(); //niveau visibilité, abstract ? statique ?  
boolean isMemberClass();  
boolean isLocalClass();  
boolean isAnonymousClass(); //vrai si classe interne anonyme  
boolean isEnum();
```

Getters sur le nom du type

```
String getSimpleName(); //nom sans le package  
String getCanonicalName(); //nom absolu (nom_package.nom_classe)  
String getName();  
String getTypeName();
```

L'API de la classe Class<T>

Getters sur l'environnement du type

```
ClassLoader getClassLoader();  
TypeVariable<Class<T>>[] getTypeParameters(); //variables de type  
Class<? super T> getSuperclass(); //getter sur la superclasse pour  
    les classes diff de Object, null sinon  
Type getGenericSuperclass();  
Package getPackage();  
Class<?>[] getInterfaces(); //liste des types des interfaces  
Type[] getGenericInterfaces();  
Class<?> getComponentType(); // type des éléments d'un tableau
```

Getters sur les attributs (type Field)

```
Field[] getFields(); //ensemble des attributs publics  
Field getField(String n); //l'attribut public de nom n  
Field[] getDeclaredFields(); //ensembles des attributs  
Field getDeclaredField(String n); //l'attribut de nom n
```

L'API de la classe `Class<T>`

Getters sur les constructeurs (`Type Constructor<T>`)

```
Constructor<?>[] getConstructors();  
Constructor<T> getConstructor(Class<?>...)  
Constructor<?>[] getDeclaredConstructors();  
Constructor<T> getDeclaredConstructor(Class<?>...)
```

Getters sur les méthodes (type `Method`)

```
Method[] getMethods() // méthodes publiques locales + héritées  
Method getMethod(String, Class<?>...)  
Method[] getDeclaredMethods() // toutes méthodes locales + héritées  
Method getDeclaredMethod(String, Class<?>...)
```

Getters sur les classes internes

```
Class<?>[] getClasses()  
Class<?>[] getDeclaredClasses()
```

Getters valides pour les classes internes

```
Method getEnclosingMethod()  
Constructor<?> getEnclosingConstructor()  
Class<?> getEnclosingClass()
```

L'API de la classe `Class<T>`

Getters sur ses annotations

```
getAnnotation(Class<A>)
isAnnotationPresent(Class<? extends Annotation>)
getAnnotationsByType(Class<A>)
getAnnotations()
getDeclaredAnnotation(Class<A>)
getDeclaredAnnotationsByType(Class<A>)
getDeclaredAnnotations()
```

Autres opérations

```
T newInstance(); //appel au constructeur par défaut
boolean isInstance(Object); //teste si un objet appartient au type
boolean isAssignableFrom(Class<?> o); //vrai si il est possible d'
    affecter une expression de type o dans une variable de type
    this
T cast(Object); // caster un objet en this
<U> Class<? extends U> asSubclass(Class<U>); //transformer un type
    T vers un sous-type de U
```


Obtenir une référence sur une classe

Cas où le type référencé est inconnu statiquement (Class<?>)

- À partir d'une référence d'un objet :
 - méthode `getClass()` offerte par la classe `Object`
 - renvoie le type réel (renseigné lors du `new`) de l'objet
⇒ ne peut ni être une interface, ni être un type primitif
- À partir du nom absolu d'un type objet en utilisant un `ClassLoader`
 - méthode `loadClass(String)` de la classe `ClassLoader`
 - la méthode statique `forName(String)` de la classe `Class` qui utilise le classloader courant.

Cas où le type référencé est connu statiquement

Faire `X.class` pour renvoyer la référence de l'objet `Class<X>` associé au type `X`.

Manipuler les membres d'un type

Trois familles de membres

- `Field` : les attributs d'un type
- `Constructor<T>` : les constructeurs
- `Method` : les méthodes

Méthodes offertes

- `Class<?> getDeclaringClass()` : référence sur le type englobant
- `String getName()` : nom du membre dans le type englobant
- `int getModifiers()` : bitset sur les modificateurs du membre.
- `boolean isAccessible()` : teste si le membre est accessible en fonction du modificateur de visibilité
- `setAccessible(boolean)` : changer l'accessibilité
⇒ **Peut permettre de forcer l'accès même si visibilité `private`**

Manipuler les membres d'un type

Méthodes communes aux Constructor et Method

- `int` `getParameterCount()` : nombre de paramètres
- `Parameter[]` `getParameters` : liste des paramètres
- `Class<?>[]` `getParameterTypes()` : liste des types des paramètres
- `TypeVariable<?>[]` `getTypeParameters()` : liste des variables de types locales

Décoder le bitset des modificateurs

Utiliser les méthodes statiques de la classe `Modifier` :

<code>isPublic(int)</code>	<code>isProtected(int)</code>	<code>isPrivate(int)</code>
<code>isAbstract(int)</code>	<code>isFinal(int)</code>	<code>isTransient(int)</code>
<code>isInterface(int)</code>	<code>isNative(int)</code>	<code>isStatic(int)</code>
<code>isSynchronized(int)</code>	<code>isVolatile(int)</code>	

Accéder aux membres (precond : `isAccessible() == true`)

Accéder à la valeur des Field

- `Object get(Object obj)` : valeur pour une instance de la classe
- `void set(Object obj, Object val)` : modifier l'attribut d'une instance de la classe

Invoquer un constructeur (`Constructor<T>`)

`T newInstance(Object ... initargs);`

Si ceci concerne une classe interne non statique, le premier argument doit être une référence de la classe englobante.

Invoquer une méthode (`Method`)

`Object invoke(Object obj, Object... args)`

- si c'est une méthode statique, `obj` est ignoré
- si la méthode retourne à la base :
 - un type primitif → version objet du type
 - `void` → `null`

Les exceptions liées au mécanisme de réflexivité

<code>ReflectiveOperationException</code>	classe mère
<code>ClassNotFoundException</code>	Impossible de charger la classe
<code>IllegalAccessException</code>	Une application tente d'accéder à un membre d'une classe alors qu'il n'est pas visible
<code>InstantiationException</code>	L'appel à <code>newInstance()</code> échoue : • Le constructeur par défaut n'existe pas • Le type représente une interface, une classe abstraite, un type primitif, un tableau
<code>InvocationTargetException</code>	Encapsule une exception jetée par l'invocation d'une méthode ou d'un constructeur.
<code>NoSuchFieldException</code>	<code>getField(String)</code> n'a pas trouvé l'attribut recherché
<code>NoSuchMethodException</code>	<code>getMethod(String, Class<?>...)</code> n'a pas trouvé la méthode recherchée <code>getConstructor(Class<?>...)</code> n'a pas trouvé le constructeur recherché

Avantages

- Manipuler des types dont le code source est inconnu
- Permet d'instancier des classes concrètes sans que le code client n'en soit dépendant
 - Factory générique : le code client peut ne dépendre que du type abstrait
- Permet d'invoquer des méthodes sans connaître leur signature exacte
 - invocation de méthodes dont les signatures respectent un certain pattern
- On peut contourner les vérifications faites par le compilateur

Inconvénients

- Risque de bug ou d'erreur à l'exécution puisque l'on perd les vérifications du compilateur
- Surcoût non négligeable en calcul.

Définition

- Objet ayant la responsabilité de charger et d'initialiser des types Java requis par la JVM au cours de l'exécution d'une application.
- Les ClassLoader d'une JVM sont organisés hiérarchiquement (père-fils)

Les trois instances préexistantes de ClassLoader dans une JVM

- le **classLoader racine (=bootstrap)** :
 - charge tous les types de base de java renseigné dans `$JAVA_HOME/jre/lib` (Object, String, ClassLoader ...)
 - Natif à la JVM et n'est pas référençable dans le code java (ref = null)
 - Son implantation dépend de l'implantation de la JVM
- Le **classloader d'extension** :
 - fils du **bootstrap** et charge tout type renseigné dans `$JAVA_HOME/lib/ext`
- Le **system classloader** ou **application classloader** :
 - fils du **classloader d'extension**
 - charge tout type renseignée dans `CLASSPATH` et `MODULEPATH`

Les sept étapes

- 1) Chargement (loading)
- 2) Liaison (linking)
- 3) Initialisation (initialization)
- 4) Instanciation (instantiation)
- 5) Récupération de la mémoire (garbage collection)
- 6) Finalisation (finalization)
- 7) Déchargement (unloading)

Objectif

Charger le bytecode du type (fichier .class) dans la mémoire de la JVM.

Comment charger un type ?

Appel à `Class<?> loadClass()` d'un `ClassLoader`

- si le type est déjà chargé et initialisé retourner l'instance `Class<?>` correspondante
- sinon on fait appel `loadClass` sur le `ClassLoader` père
 - si le père a réussi, retourner l'instance `Class<?>`
 - si échec, faire appel à `findClass()`
 - si il trouve un fichier .class dont il a responsabilité alors
 - 1) charger le fichier en mémoire
 - 2) passer à l'étape de liaison
 - sinon `throw ClassNotFoundException`

Objectif

- 1) Vérifier la compatibilité du bytecode du type dans la JVM
- 2) Allouer la mémoire nécessaire à la classe
- 3) Transformer les symboles du constant pool en référence mémoire

Vérifications

- Instructions ByteCode (instructions valides ?)
- Déclarations d'entités du constant pool (valeurs constantes)
- Le super-type racine est `Object`.
- Les classes finales n'ont pas de classes filles
- Les méthodes finales ne sont pas réécrites
- Les méthodes des interfaces implémentées sont définies
- Deux méthodes n'ont pas la même signature

Objectif

Exécuter le code d'initialisation d'un type

Étapes

- Si il existe un type père qui n'a pas été initialisé
initialiser le type père
- Initialiser tous les attributs statiques
- exécuter tous les blocs statiques si ils existent

Quand la phase d'initialisation est-elle déclenchée pour un type ?

- À la création d'une nouvelle instance en utilisant l'opérateur new
- À la création d'un tableau dont les composants sont du type cible
- Utilisation d'un sous-type (on initialise toujours les super-types avant)